Indian Startup Funding

Exploratory Data Analysis — Step-by-Step Guide

2018 · 2019 · 2020 · 2021

Dataset	4 CSV files (2018–2021) | 2,879 total startup funding records
Goal	EDA to support ML modelling on the Indian startup ecosystem
Approach	Google Colab · display() · pandas · seaborn · matplotlib
Audience	Young entrepreneurs & data analysts seeking funding insights

KEY BUSINESS QUESTIONS THIS EDA ANSWERS

- Q1 · Who is the biggest investor in the Indian startup ecosystem?
- Q2 · How has the funding ecosystem changed with time (2018–2021)?
- Q3 · Do cities play a major role in attracting startup funding?
- Q4 · Which industries / sectors are most favored by investors?

- Q5 · What funding stages dominate the ecosystem?

Table of Contents

PART 1 – Setup & Environment
- 1.1 Installing Libraries
- 1.2 Importing Libraries

PART 2 – Data Loading & Initial Exploration
- 2.1 Loading All Four Datasets
- 2.2 Previewing Each Dataset — .head() & .info()
- 2.3 Shape and Schema Overview
- 2.4 Standardising Column Names

PART 3 – Data Cleaning
- 3.1 Handling Missing Values
- 3.2 Dropping Irrelevant Columns
- 3.3 Fixing the Amount Column (currency, symbols, 'Undisclosed')
- 3.4 Standardising Location / City Column
- 3.5 Standardising Sector / Industry Column
- 3.6 Merging All Years with a Year Flag

PART 4 – Descriptive / Univariate Analysis
- 4.1 Summary Statistics (.describe())
- 4.2 Funding Amount Distribution
- 4.3 Funding Stage Distribution
- 4.4 Top Cities / HQs
- 4.5 Top Sectors
- 4.6 Top Investors

PART 5 – Bivariate Analysis
- 5.1 Funding Amount vs. Stage
- 5.2 Funding Amount vs. City
- 5.3 Funding Amount vs. Sector
- 5.4 Funding Count Over Time (Year)
- 5.5 Stage vs. Sector (Cross-tab)

PART 6 – Multivariate Analysis
- 6.1 Correlation Heatmap
- 6.2 Pairplot of Numeric Features
- 6.3 Funding Amount by City & Stage (Grouped Bar)
- 6.4 Investor Activity by Sector & Year (Pivot Heatmap)
- 6.5 Treemap — Sector Share of Total Funding

PART 7 – Answering the Business Questions
- Q1 · Biggest Investor(s)
- Q2 · Ecosystem Change Over Time
- Q3 · City Impact on Funding
- Q4 · Favored Industries
- Q5 · Dominant Funding Stages

PART 8 – Preparing Data for ML
- 8.1 Feature Engineering
- 8.2 Encoding Categorical Variables
- 8.3 Handling Class Imbalance
- 8.4 Train / Validation / Test Split

DATASET OVERVIEW

Understanding Your Raw Data

■■ Before writing a single line of EDA code, it is critical to understand what each file contains. The four CSVs cover funding rounds recorded in the Indian startup ecosystem from 2018 to 2021. They do NOT share the same schema — 2018 uses different column names — so merging them requires careful standardisation.

Company Name	Company/Brand	Startup name	Key identifier
Industry	Sector	Business domain	Many duplicates/variants
Round/Series	Stage	Funding round type	~35–52% null in 19–21
Amount	Amount($)	Capital raised	Mixed ■/$, 'Undisclosed'
Location	HeadQuarter	City / State / Country	Needs city extraction
N/A	Investor	Investor name(s)	Often multiple per row
N/A	Founded	Year founded	Useful for age feature
N/A	Founders	Founder name(s)	Not used in EDA core

File	Rows	Columns	Key Null Columns
startup_funding2018.csv	526	6	None (but '—' used as placeholder for missing amounts)
startup_funding2019.csv	89	9	Stage (51.7%), Founded (32.6%), HeadQuarter (21.4%)
startup_funding2020.csv	1,055	10	Stage (44.0%), Founded (20.1%), Unnamed:9 (99.8%)
startup_funding2021.csv	1,209	9	Stage (35.4%), Investor (5.1%)

PART 1

Setup & Environment

1.1 Installing Required Libraries

Run this in the very first cell of your Colab notebook. The **!** prefix tells Colab to execute the command in the underlying Linux shell rather than as Python. We install only what pandas/matplotlib do not already ship with in Colab.

```
# CELL 1 — Install extra libraries
# squarify → treemap visualisations
# missingno → visual missing-value maps
# plotly → interactive charts (optional)
!pip install squarify missingno plotly --quiet
```

■ **Why these?** squarify draws treemaps that make sector share intuitive at a glance. missingno renders missing-value matrices so you can spot patterns (e.g., whole columns being blank) in seconds instead of reading numbers. plotly makes charts interactive for deeper exploration in a notebook environment.

1.2 Importing All Libraries

Group imports logically so the notebook is easy to maintain. We set the matplotlib style and default figure size here once so every subsequent plot is consistent.

```
# CELL 2 — Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import squarify
import missingno as msno
import warnings
from IPython.display import display, HTML

warnings.filterwarnings('ignore') # suppress noisy warnings

# ■■ Aesthetics ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
plt.style.use('seaborn-v0_8-whitegrid') # clean grid style
sns.set_palette('tab10') # colorblind-friendly palette
pd.set_option('display.max_columns', 20) # show wide DataFrames fully
```

```
pd.set_option('display.float_format', '{:,.2f}'.format)
```

We use display() throughout instead of print() because in Jupyter/Colab it renders DataFrames as rich HTML tables, plots inline, and HTML styled strings — giving a much more readable notebook.

PART 2

Data Loading & Initial Exploration

2.1 Loading All Four Datasets

We load each file individually before merging so we can inspect and clean them separately. Passing **dtype=str** for the Amount column prevents pandas from silently converting currency strings to NaN.

```
# CELL 3 — Load raw data

df18 = pd.read_csv('startup_funding2018.csv')

df19 = pd.read_csv('startup_funding2019.csv')

df20 = pd.read_csv('startup_funding2020.csv')

df21 = pd.read_csv('startup_funding2021.csv')


display(HTML('2018 — first 3 rows'))

display(df18.head(3))


display(HTML('2021 — first 3 rows'))

display(df21.head(3))
```

Reason: We display.head() on the extreme years (2018 and 2021) side-by-side to immediately see the column name difference and the different currency formats in the Amount field.

2.2 Previewing Each Dataset — .head() & .info()

```
# CELL 4 — Shape + dtypes for all files

for name, df in [('2018', df18), ('2019', df19),

('2020', df20), ('2021', df21)]:

display(HTML(f'■■ {name} Dataset ■■'))

display(HTML(f'Shape: {df.shape[0]} rows × {df.shape[1]} cols'))

display(df.dtypes.to_frame('dtype').T) # compact dtype view

display(df.head(3))
```

Reason: .dtypes tells us which columns pandas read as object (string) vs numeric. The Amount column reads as object across all files because it contains currency symbols — we must handle this manually in the cleaning step.

2.3 Visual Missing-Value Map

```
# CELL 5 — Missing-value matrix (visual)

fig, axes = plt.subplots(2, 2, figsize=(14, 8))

pairs = [('2018', df18), ('2019', df19), ('2020', df20), ('2021', df21)]


for ax, (yr, df) in zip(axes.flatten(), pairs):
```

```
msno.matrix(df, ax=ax, sparkline=False, fontsize=9)
ax.set_title(f'{yr} — Missing Value Matrix', fontsize=11, fontweight='bold')

plt.suptitle('White = Data Present | Black = Missing',
fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.savefig('missing_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
```

Reason: missingno renders white stripes where data exists and black where it is absent. Patterns (e.g., a whole column of black) reveal structural missingness — meaning the column was never collected that year — which changes how we handle it versus random missingness.

2.4 Null Count Summary Table

```
# CELL 6 — Null counts as styled DataFrame
def null_report(df, year):
null_df = pd.DataFrame({
'Column' : df.columns,
'Null Count' : df.isnull().sum().values,
'Null %' : (df.isnull().sum() / len(df) * 100).round(2).values,
'Dtype' : df.dtypes.values,
})
display(HTML(f'{year} — Null Report'))
display(null_df.style.background_gradient(subset=['Null %'],
cmap='OrRd'))

null_report(df18, '2018')
null_report(df19, '2019')
null_report(df20, '2020')
null_report(df21, '2021')
```

Reason: The gradient heatmap lets you instantly spot the most problematic columns (deep red = high null %). For instance, Stage is missing in 35–52% of rows across years — this is important context for deciding whether to impute or drop.

PART 3

Data Cleaning

■■ Data cleaning is the single most important step before any analysis. Every decision here is documented with a reason so you can justify your choices when presenting your findings.

3.1 Standardising Column Names Across All Years

2018 uses completely different column names. We rename them to match 2019–2021 so we can concat all four DataFrames into one.

```
# CELL 7 — Rename 2018 columns to match 2019-2021 schema
df18 = df18.rename(columns={
'Company Name' : 'Company/Brand',
'Industry' : 'Sector',
'Round/Series' : 'Stage',
'Amount' : 'Amount($)',
'Location' : 'HeadQuarter',
'About Company': 'What it does',
})

# Add missing columns so all four share the same schema
df18['Founded'] = np.nan
df18['Founders'] = np.nan
df18['Investor'] = np.nan

display(HTML('2018 columns after rename:'))
display(df18.columns.tolist())
```

Reason: pd.concat() aligns on column names. If 2018 still has 'Company Name' while 2021 has 'Company/Brand', concat creates two separate columns — one filled with NaN for the other years. Renaming before concat avoids this silently broken merge.

3.2 Dropping Irrelevant / Redundant Columns

```
# CELL 8 — Drop Unnamed:9 from 2020 (99.8% null, no information)
df20 = df20.drop(columns=['Unnamed: 9'], errors='ignore')

# Verify
display(HTML('2020 columns after drop:'))
display(df20.columns.tolist())
```

Reason: A column that is 99.8% null provides no analytical or predictive value. Including it would create noise in every aggregation and slow down operations. errors='ignore' makes the code safe — it simply skips the drop if the column doesn't exist.

3.3 Cleaning the Amount Column (The Hardest Step)

■■ The Amount column is the messiest in this dataset. It contains: USD amounts ($1,200,000), INR amounts (■40,000,000), plain integers (250000), the string 'Undisclosed', and '—' as a placeholder for missing values. We need one clean numeric column in USD.

```
# CELL 9 — Clean Amount column and convert to USD (millions)

# Exchange rate approximation used (historical average for the period)

INR_TO_USD = 0.013 # 1 INR ≈ 0.013 USD (approx. 2018–2021 average)

def clean_amount(val):

"""

Convert any amount string to a float in USD.

Returns np.nan for undisclosed or truly missing values.

"""

if pd.isna(val): # already NaN

return np.nan

val = str(val).strip()

if val in ['Undisclosed', 'undisclosed', # known placeholders

'—', '-', 'N/A', '']:

return np.nan

is_inr = '■' in val # Indian Rupee flag

# Strip all non-numeric characters except the decimal point

val = val.replace('■', '').replace('$', '')

.replace(',', '').strip()

try:

amount = float(val)

except ValueError:

return np.nan

return round(amount * INR_TO_USD, 2) if is_inr else amount

# Apply to all four DataFrames

for df in [df18, df19, df20, df21]:

df['Amount_USD'] = df['Amount($)'].apply(clean_amount)

df['Amount_M'] = df['Amount_USD'] / 1_000_000 # express in millions
```

```
# Verify: show null counts and descriptive stats
display(HTML('Amount_USD — Null counts by year'))
for name, df in [('2018', df18), ('2019', df19),
('2020', df20), ('2021', df21)]:
n_null = df['Amount_USD'].isna().sum()
pct = round(n_null / len(df) * 100, 1)
display(HTML(f'{name}: {n_null} nulls ({pct}%)'))
```

Reason: We write a reusable function so the same logic applies consistently across all four files. Converting to USD puts all amounts on the same scale for comparison. Dividing by 1,000,000 to get millions (Amount_M) makes axis labels on charts readable. We never drop rows just because Amount is NaN — we only drop NaN when the specific analysis requires a numeric value.

3.4 Standardising the City Column

The HeadQuarter column often contains full addresses like 'Bangalore, Karnataka, India'. We extract only the first token (city).

```
# CELL 10 — Extract city from HeadQuarter
def extract_city(val):
"""Take only the first comma-separated segment as the city name."""
if pd.isna(val):
return np.nan
city = str(val).split(',')[0].strip()
return city if city else np.nan

for df in [df18, df19, df20, df21]:
df['City'] = df['HeadQuarter'].apply(extract_city)

# Standardise common variants
city_map = {
'Gurugram' : 'Gurgaon',
'New Delhi' : 'Delhi',
'Bengaluru' : 'Bangalore',
'Ahmadabad' : 'Ahmedabad',
}
for df in [df18, df19, df20, df21]:
df['City'] = df['City'].replace(city_map)

display(HTML('Top 10 Cities (2021)'))
```

```
display(df21['City'].value_counts().head(10).to_frame())
```

Reason: Without standardisation, 'Gurugram' and 'Gurgaon' appear as two different cities in your charts — splitting what is actually one cluster. This is called entity disambiguation and is a standard NLP/data-cleaning technique.

3.5 Standardising the Sector Column

```
# CELL 11 — Collapse duplicate sector labels

sector_map = {

'Edtech' : 'EdTech',

'E-learning' : 'EdTech',

'Financial Services' : 'FinTech',

'HealthCare' : 'Healthcare',

'HealthTech' : 'Healthcare',

'Health, Wellness & Fitness': 'Healthcare',

'E-commerce' : 'E-Commerce',

'Ecommerce' : 'E-Commerce',

'Logistics' : 'Logistics & Supply Chain',

'SaaS startup' : 'SaaS',

'Computer Software' : 'SaaS',

'Information Technology & Services': 'IT & Services',

}

for df in [df18, df19, df20, df21]:

df['Sector'] = df['Sector'].replace(sector_map)

df['Sector'] = df['Sector'].str.strip() # remove whitespace


display(HTML('Top 15 Sectors after standardisation (2021)'))

display(df21['Sector'].value_counts().head(15).to_frame())
```

Reason: The raw data has the same concept labelled inconsistently ('Edtech', 'EdTech', 'E-learning' are all the same). Without collapsing these, your bar chart shows tiny separate bars for each variant instead of one tall bar for 'EdTech' — misleading results.

3.6 Merging All Four Years into One DataFrame

```
# CELL 12 — Tag each year, then concatenate

df18['Year'] = 2018

df19['Year'] = 2019

df20['Year'] = 2020

df21['Year'] = 2021
```

```
# Columns we keep for analysis
KEEP = ['Company/Brand', 'Sector', 'Stage', 'Amount_USD',
'Amount_M', 'City', 'Investor', 'Founded', 'Year']

df = pd.concat([df18[KEEP], df19[KEEP],
df20[KEEP], df21[KEEP]],
ignore_index=True)

display(HTML('Combined DataFrame'))
display(HTML(f'Total records: {len(df):,}'))
display(df.info())
display(df.head(5))
```

Reason: ignore_index=True resets the index so it runs 0 … 2878 continuously. Without it you would get repeated indices (0-525, 0-88, etc.) which breaks groupby and loc-based indexing. The Year column is our primary temporal variable for all time-series analyses.

PART 4

Descriptive / Univariate Analysis

Univariate analysis examines one variable at a time to understand its distribution, central tendency, and spread. Start here before looking at relationships between variables.

4.1 Summary Statistics — .describe()

```
# CELL 13 — Descriptive statistics

display(HTML('Numeric Summary'))

display(df[['Amount_M', 'Founded', 'Year']]

.describe()

.style.format('{:,.2f}')

.background_gradient(cmap='Blues'))

display(HTML('Categorical Summary'))

display(df[['Sector', 'Stage', 'City']].describe())
```

Reason: .describe() gives count, mean, std, min, 25th/50th/75th percentile, and max. The huge gap between the 75th percentile and the max of Amount_M will reveal heavy right-skew — confirming we should use log scale in plots and consider log-transformation before ML modelling.

4.2 Funding Amount Distribution

```
# CELL 14 — Distribution of funding amounts (log scale)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

amt = df['Amount_M'].dropna()

# Left: raw histogram (shows extreme skew)

axes[0].hist(amt, bins=60, color='#3B82F6', edgecolor='white', alpha=0.8)

axes[0].set_title('Raw Amount Distribution', fontsize=13, fontweight='bold')

axes[0].set_xlabel('Amount ($ Millions)')

axes[0].set_ylabel('Frequency')

# Right: log-scale histogram (reveals the true shape)

axes[1].hist(np.log1p(amt), bins=60, color='#1B7F79',

edgecolor='white', alpha=0.8)

axes[1].set_title('Log-Transformed Amount (log1p)', fontsize=13, fontweight='bold')

axes[1].set_xlabel('log(1 + Amount in $ Millions)')

axes[1].set_ylabel('Frequency')
```

```
plt.suptitle('Funding Amount Distribution — All Years',

fontsize=14, fontweight='bold')

plt.tight_layout()

plt.savefig('amount_distribution.png', dpi=150, bbox_inches='tight')

plt.show()

# Key statistics

display(HTML(f'''

**Median Funding:** ${amt.median():,.2f}M  |

**Mean:** ${amt.mean():,.2f}M  |

**Max:** ${amt.max():,.2f}M

'''))
```

Reason: Most startups raise small amounts while a few raise hundreds of millions — a classic power-law distribution. Showing both raw and log-transformed histograms illustrates this skew clearly. log1p is used (instead of log) because it handles zero-amount rows safely: log1p(0) = 0 while log(0) = -inf.

4.3 Funding Stage Distribution

```
# CELL 15 — Stage distribution

stage_counts = (df['Stage']

.dropna()

.value_counts()

.head(12))

fig, ax = plt.subplots(figsize=(12, 5))

bars = ax.barh(stage_counts.index[::-1], stage_counts.values[::-1],

color=sns.color_palette('tab10', 12))

# Annotate bars with counts

for bar in bars:

ax.text(bar.get_width() + 5, bar.get_y() + bar.get_height()/2,

f'{int(bar.get_width()):,}', va='center', fontsize=9)

ax.set_title('Funding Stage Distribution — All Years (2018–2021)',

fontsize=13, fontweight='bold')

ax.set_xlabel('Number of Deals')

ax.set_ylabel('Stage')

plt.tight_layout()

plt.savefig('stage_distribution.png', dpi=150, bbox_inches='tight')
```

```
plt.show()

display(stage_counts.to_frame('Count'))
```

Reason: Horizontal bar charts are easier to read when category labels are long strings. Reversing the order ([::-1]) places the largest bar at the top — the natural reading direction. We use head(12) to focus on meaningful stages and avoid visual clutter from data-entry errors like '$1200000' appearing in the Stage column.

4.4 Top Cities / Headquarters

```
# CELL 16 — Top 10 cities by deal count
city_counts = df['City'].value_counts().head(10)

fig, ax = plt.subplots(figsize=(10, 5))
ax.bar(city_counts.index, city_counts.values,
color=sns.color_palette('mako_r', 10), edgecolor='white')
ax.set_title('Top 10 Cities by Number of Startup Deals (2018–2021)',
fontsize=13, fontweight='bold')
ax.set_xlabel('City')
ax.set_ylabel('Number of Deals')
plt.xticks(rotation=30, ha='right')

for i, v in enumerate(city_counts.values):
ax.text(i, v + 3, str(v), ha='center', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('top_cities.png', dpi=150, bbox_inches='tight')
plt.show()
```

4.5 Top Sectors / Industries

```
# CELL 17 — Top 15 sectors by deal count
sector_counts = df['Sector'].value_counts().head(15)

fig, ax = plt.subplots(figsize=(12, 5))
sns.barplot(x=sector_counts.values, y=sector_counts.index,
palette='viridis', ax=ax)
ax.set_title('Top 15 Sectors by Number of Deals (2018–2021)',
fontsize=13, fontweight='bold')
ax.set_xlabel('Number of Deals')
ax.set_ylabel('Sector')
```

```
for i, v in enumerate(sector_counts.values):
ax.text(v + 1, i, str(v), va='center', fontsize=9)

plt.tight_layout()
plt.savefig('top_sectors.png', dpi=150, bbox_inches='tight')
plt.show()
```

4.6 Top Investors

The Investor column often contains multiple investors per deal separated by commas. We need to explode these to count each investor individually.

```
# CELL 18 — Top investors (explode multi-investor rows)
investor_series = (df['Investor']
.dropna()
.str.split(',') # split by comma
.explode() # one investor per row
.str.strip() # remove whitespace
.str.title() # normalise capitalisation
)

top_investors = investor_series.value_counts().head(15)

fig, ax = plt.subplots(figsize=(12, 6))
sns.barplot(x=top_investors.values, y=top_investors.index,
palette='rocket_r', ax=ax)
ax.set_title('Top 15 Investors by Number of Deals — Indian Startup Ecosystem',
fontsize=13, fontweight='bold')
ax.set_xlabel('Number of Deals')
ax.set_ylabel('Investor')

for i, v in enumerate(top_investors.values):
ax.text(v + 0.3, i, str(v), va='center', fontsize=9, fontweight='bold')

plt.tight_layout()
plt.savefig('top_investors.png', dpi=150, bbox_inches='tight')
plt.show()

display(HTML('Top 15 Investors'))
display(top_investors.to_frame('Deal Count'))
```

Reason: Without .explode(), a deal with 'Sequoia Capital, BEENEXT' would count as one entry for that combined string — so Sequoia Capital would appear to have far fewer deals than it actually does. .str.title() normalises 'sequoia capital' and 'Sequoia Capital' to the same canonical form.

PART 5

Bivariate Analysis

Bivariate analysis examines the relationship between two variables. Every chart here directly addresses one of the five business questions.

5.1 Funding Amount vs. Stage (Box Plot)

Business Question addressed: What funding stages attract the most capital?

```
# CELL 19 — Box plot: Amount vs Stage

# Focus on the 8 most common stages to keep chart readable

top_stages = df['Stage'].value_counts().head(8).index

df_stage = df[df['Stage'].isin(top_stages)].copy()

# Use log1p transform on y-axis to handle extreme outliers

df_stage['log_Amount'] = np.log1p(df_stage['Amount_M'])

# Order stages by median funding (ascending) for visual clarity

order = (df_stage.groupby('Stage')['log_Amount']

.median()

.sort_values(ascending=False)

.index)

fig, ax = plt.subplots(figsize=(13, 6))

sns.boxplot(data=df_stage, x='Stage', y='log_Amount',

order=order, palette='Set2', ax=ax)

sns.stripplot(data=df_stage, x='Stage', y='log_Amount',

order=order, color='black', alpha=0.3, size=2.5, ax=ax)

ax.set_title('Funding Amount by Stage (log scale) — 2018–2021',

fontsize=13, fontweight='bold')

ax.set_xlabel('Funding Stage')

ax.set_ylabel('log(1 + Amount in $M)')

plt.xticks(rotation=20, ha='right')

plt.tight_layout()

plt.savefig('amount_vs_stage.png', dpi=150, bbox_inches='tight')

plt.show()
```

Reason: Box plots show median, IQR, and outliers simultaneously. We overlay strip plots (individual points) so you can see how many data points back up each box — a box with only 5 data points is less reliable than

one with 200. Sorting by median funding amount creates a natural ranking of stages.

5.2 Funding Amount vs. City

Business Question addressed: Do cities play a major role in funding?

```
# CELL 20 — Total funding raised per city

top_cities = df['City'].value_counts().head(10).index

city_funding = (df[df['City'].isin(top_cities)]

.groupby('City')['Amount_M']

.agg(['sum', 'median', 'count'])

.rename(columns={'sum':'Total_M',

'median':'Median_M',

'count':'Deals'})

.sort_values('Total_M', ascending=False))

display(HTML('City-Level Funding Summary'))

display(city_funding.style.format({'Total_M': '${:,.1f}M',

'Median_M': '${:,.2f}M',

'Deals': '{:,}'}))

fig, ax = plt.subplots(figsize=(11, 5))

ax.bar(city_funding.index, city_funding['Total_M'],

color=sns.color_palette('crest', len(city_funding)),

edgecolor='white')

ax.set_title('Total Funding Raised per City ($M) — 2018–2021',

fontsize=13, fontweight='bold')

ax.set_ylabel('Total Funding ($M)')

ax.set_xlabel('City')

plt.xticks(rotation=30, ha='right')

plt.tight_layout()

plt.savefig('city_funding.png', dpi=150, bbox_inches='tight')

plt.show()
```

5.3 Funding Amount vs. Sector

Business Question addressed: Which industries are favored by investors?

```
# CELL 21 — Total and median funding by sector

top_sectors = df['Sector'].value_counts().head(12).index

sector_agg = (df[df['Sector'].isin(top_sectors)]
```

```
    .groupby('Sector')['Amount_M']
    .agg(['sum', 'median', 'count'])
    .rename(columns={'sum':'Total_M',
    'median':'Median_M',
    'count':'Deals'})
    .sort_values('Total_M', ascending=False))

    fig, axes = plt.subplots(1, 2, figsize=(16, 5))

    # Left: Total funding
    axes[0].barh(sector_agg.index[::-1], sector_agg['Total_M'][::-1],
    color=sns.color_palette('plasma', len(sector_agg)))
    axes[0].set_title('Total Funding by Sector ($M)', fontsize=12, fontweight='bold')
    axes[0].set_xlabel('Total Funding ($M)')

    # Right: Median deal size
    axes[1].barh(sector_agg.index[::-1], sector_agg['Median_M'][::-1],
    color=sns.color_palette('viridis', len(sector_agg)))
    axes[1].set_title('Median Deal Size by Sector ($M)', fontsize=12,
    fontweight='bold')
    axes[1].set_xlabel('Median Deal Size ($M)')

    plt.suptitle('Sector Funding Analysis — Total vs Median (2018–2021)',
    fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.savefig('sector_funding.png', dpi=150, bbox_inches='tight')
    plt.show()
```

Reason: Total funding shows which sectors attracted the most capital in absolute terms (good for identifying dominant sectors). Median deal size shows which sectors get large individual cheques on average (good for identifying where big investors focus). Showing both side by side prevents misleading conclusions — a sector with many tiny deals may rank highly on total but low on median.

5.4 Funding Count & Amount Over Time

Business Question addressed: How has the funding ecosystem changed from 2018 to 2021?

```
    # CELL 22 — Year-over-year trends
    yearly = df.groupby('Year').agg(
    Deals = ('Company/Brand', 'count'),
    Total_M = ('Amount_M', 'sum'),
```

```
Median_M = ('Amount_M', 'median'),
Unique_Inv = ('Investor', 'nunique'),
).reset_index()

display(HTML('Year-over-Year Summary'))
display(yearly.style.format({
'Total_M' : '${:,.1f}M',
'Median_M' : '${:,.2f}M',
'Deals' : '{:,}',
'Unique_Inv': '{:,}',
}))

fig, axes = plt.subplots(2, 2, figsize=(14, 8))

metrics = [('Deals', 'Number of Deals', '#3B82F6'),
('Total_M', 'Total Funding ($M)', '#1B7F79'),
('Median_M', 'Median Deal Size ($M)', '#E8A838'),
('Unique_Inv', 'Unique Investors', '#7C3AED')]

for ax, (col, label, color) in zip(axes.flatten(), metrics):
ax.plot(yearly['Year'], yearly[col], marker='o',
color=color, linewidth=2.5, markersize=8)
ax.fill_between(yearly['Year'], yearly[col],
alpha=0.15, color=color)
ax.set_title(label, fontsize=11, fontweight='bold')
ax.set_xticks(yearly['Year'])
ax.set_xlabel('Year')
ax.set_ylabel(label)

plt.suptitle('Indian Startup Ecosystem — Year-over-Year Trends',
fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('yoy_trends.png', dpi=150, bbox_inches='tight')
plt.show()
```

5.5 Stage vs. Sector Cross-Tabulation

```
# CELL 23 — Cross-tab heatmap: Stage vs Sector
top_s = df['Stage'].value_counts().head(7).index
```

```
top_sec = df['Sector'].value_counts().head(10).index

ct = pd.crosstab(
df[df['Stage'].isin(top_s)]['Stage'],
df[df['Sector'].isin(top_sec)]['Sector'],
)

fig, ax = plt.subplots(figsize=(14, 5))
sns.heatmap(ct, annot=True, fmt='d', cmap='YlOrRd',
linewidths=0.5, linecolor='white', ax=ax)
ax.set_title('Deal Count: Funding Stage vs Sector',
fontsize=13, fontweight='bold')
ax.set_xlabel('Sector')
ax.set_ylabel('Stage')
plt.xticks(rotation=35, ha='right')
plt.tight_layout()
plt.savefig('stage_sector_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```

Reason: pd.crosstab() creates a frequency matrix — how many deals fall in each Stage × Sector combination. The heatmap makes hot-spots immediately visible (e.g., Seed-stage deals dominating EdTech). This is crucial for a young entrepreneur choosing a sector — it shows not just which sectors are popular but at which stage investors enter.

PART 6

Multivariate Analysis

Multivariate analysis studies three or more variables simultaneously to uncover patterns that bivariate analysis misses.

6.1 Correlation Heatmap of Numeric Features

```
# CELL 24 — Feature engineering for correlation analysis
# We need numeric columns — encode categoricals as label-encoded integers
from sklearn.preprocessing import LabelEncoder

df_corr = df[['Amount_M', 'Year', 'Sector', 'Stage', 'City']].copy()

le = LabelEncoder()
for col in ['Sector', 'Stage', 'City']:
df_corr[col + '_enc'] = le.fit_transform(df_corr[col].fillna('Unknown'))

numeric_cols = ['Amount_M', 'Year',
'Sector_enc', 'Stage_enc', 'City_enc']

corr = df_corr[numeric_cols].corr()

fig, ax = plt.subplots(figsize=(8, 6))
mask = np.triu(np.ones_like(corr, dtype=bool)) # upper triangle mask
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
center=0, linewidths=0.5, square=True, ax=ax,
vmin=-1, vmax=1)
ax.set_title('Correlation Matrix — Numeric Features',
fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```

Reason: The upper-triangle mask prevents showing every correlation twice (a matrix is symmetric). coolwarm diverging colormap centres on zero — blue = negative correlation, red = positive. Note: correlations with label-encoded categoricals are ordinally arbitrary and should be interpreted with caution — they are included here for feature selection inspiration, not as definitive relationships.

6.2 Pairplot of Key Numeric Features

```
# CELL 25 — Pairplot (sample 500 rows to keep it fast)
```

```
df_pair = df[['Amount_M', 'Year', 'Stage']].dropna().copy()
df_pair['log_Amount'] = np.log1p(df_pair['Amount_M'])

# Sample for speed — pairplot on 2,000+ rows is slow
sample = df_pair.sample(min(500, len(df_pair)), random_state=42)

top_stages_pair = df['Stage'].value_counts().head(5).index
sample = sample[sample['Stage'].isin(top_stages_pair)]

g = sns.pairplot(sample[['log_Amount', 'Year', 'Stage']],
hue='Stage', palette='tab10',
plot_kws={'alpha': 0.6, 's': 30})
g.figure.suptitle('Pairplot — log(Amount), Year, Stage',
y=1.02, fontsize=13, fontweight='bold')
plt.savefig('pairplot.png', dpi=120, bbox_inches='tight')
plt.show()
```

6.3 Funding Amount by City & Funding Stage (Grouped Bar)

```
# CELL 26 — Grouped bar: City × Stage
top_c = df['City'].value_counts().head(5).index
top_st = ['Seed', 'Series A', 'Series B', 'Series C', 'Pre-seed']

pivot = (df[df['City'].isin(top_c) & df['Stage'].isin(top_st)]
.groupby(['City', 'Stage'])['Amount_M']
.sum()
.unstack(fill_value=0))

pivot.plot(kind='bar', figsize=(13, 6), colormap='tab10',
edgecolor='white', width=0.75)
plt.title('Total Funding by City & Stage ($M) — 2018–2021',
fontsize=13, fontweight='bold')
plt.xlabel('City')
plt.ylabel('Total Funding ($M)')
plt.xticks(rotation=0)
plt.legend(title='Stage', bbox_to_anchor=(1.01, 1), loc='upper left')
plt.tight_layout()
plt.savefig('city_stage_grouped.png', dpi=150, bbox_inches='tight')
plt.show()
```

6.4 Investor Activity by Sector & Year (Pivot Heatmap)

```
# CELL 27 — Sector × Year deal-count heatmap
top_sec_list = df['Sector'].value_counts().head(10).index

pivot_sy = (df[df['Sector'].isin(top_sec_list)]
.groupby(['Sector', 'Year'])
.size()
.unstack(fill_value=0))

fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(pivot_sy, annot=True, fmt='d', cmap='Blues',
linewidths=0.5, linecolor='white', ax=ax)
ax.set_title('Deal Count by Sector & Year',
fontsize=13, fontweight='bold')
ax.set_xlabel('Year')
ax.set_ylabel('Sector')
plt.tight_layout()
plt.savefig('sector_year_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```

Reason: This three-way analysis (sector, year, deal count) simultaneously shows which sectors are booming and when the boom happened. For example, a dark blue cell at EdTech-2021 versus a light cell at EdTech-2018 would visually confirm the COVID-19-driven EdTech surge.

6.5 Treemap — Sector Share of Total Funding

```
# CELL 28 — Treemap of sector funding share
sector_total = (df.groupby('Sector')['Amount_M']
.sum()
.sort_values(ascending=False)
.head(15))

fig, ax = plt.subplots(figsize=(14, 7))
squarify.plot(
sizes = sector_total.values,
label = [f'{s}\n${v:,.0f}M'
for s, v in zip(sector_total.index, sector_total.values)],
color = sns.color_palette('tab20', len(sector_total)),
alpha = 0.85,
```

```
    text_kwargs = {'fontsize': 9, 'fontweight': 'bold', 'color': 'white'},
    ax = ax,
)
ax.set_title('Treemap — Total Funding Share by Sector ($M) 2018–2021',
fontsize=14, fontweight='bold')
ax.axis('off')
plt.tight_layout()
plt.savefig('treemap_sectors.png', dpi=150, bbox_inches='tight')
plt.show()
```

Reason: A treemap encodes two dimensions — the number of sectors and their relative sizes — in a single compact chart. The area of each rectangle is proportional to total funding, making the dominance of top sectors immediately apparent without needing to read axis values.

PART 7

Answering the Business Questions

■ This section synthesises everything from Parts 4–6 into direct, data-driven answers to the five business questions. Run these cells after all cleaning and analysis cells are complete.

Q1 · Who is the biggest investor in the Indian startup ecosystem?

```
# CELL 29 — Biggest investors: by deal count AND by total capital deployed
inv_exploded = (df[['Investor', 'Amount_M']]
.dropna(subset=['Investor'])
.assign(Investor=lambda x: x['Investor'].str.split(','))
.explode('Investor')
.assign(Investor=lambda x: x['Investor'].str.strip().str.title()))

# Metric 1: Number of deals
by_deals = inv_exploded['Investor'].value_counts().head(10)

# Metric 2: Total capital deployed
by_capital = (inv_exploded.groupby('Investor')['Amount_M']
.sum()
.sort_values(ascending=False)
.head(10))

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Left: by deals
sns.barplot(x=by_deals.values, y=by_deals.index,
palette='rocket_r', ax=axes[0])
axes[0].set_title('Top 10 Investors — By Number of Deals',
fontsize=12, fontweight='bold')
axes[0].set_xlabel('Number of Deals')

# Right: by capital
sns.barplot(x=by_capital.values, y=by_capital.index,
palette='mako_r', ax=axes[1])
axes[1].set_title('Top 10 Investors — By Total Capital Deployed ($M)',
fontsize=12, fontweight='bold')
```

```
axes[1].set_xlabel('Total Capital Deployed ($M)')

plt.suptitle('Q1: Who is the Biggest Investor?',

fontsize=14, fontweight='bold')

plt.tight_layout()

plt.savefig('Q1_biggest_investor.png', dpi=150, bbox_inches='tight')

plt.show()

display(HTML('By Deals'))

display(by_deals.to_frame('Deals'))

display(HTML('By Capital ($M)'))

display(by_capital.to_frame('Total_M').style.format('${:,.1f}M'))
```

Note: Two metrics are used deliberately — the most active investor (by deal count) is not always the biggest capital deployer. Angel investors and accelerators (like Y Combinator) make many small bets while VCs like Tiger Global make fewer but much larger investments.

Q2 · How has the funding ecosystem changed from 2018 to 2021?

```
# CELL 30 — Ecosystem change: growth metrics year over year

yoy = df.groupby('Year').agg(

Deals = ('Company/Brand', 'count'),

Total_M = ('Amount_M', 'sum'),

Median_M = ('Amount_M', 'median'),

Unique_Sec = ('Sector', 'nunique'),

).reset_index()

# Calculate YoY growth rates

yoy['Deal_Growth%'] = yoy['Deals'].pct_change() * 100

yoy['Total_Growth%'] = yoy['Total_M'].pct_change() * 100

display(HTML('Q2 — Year-over-Year Ecosystem Metrics'))

display(yoy.style.format({

'Total_M' : '${:,.1f}M',

'Median_M' : '${:,.2f}M',

'Deal_Growth%' : '{:+.1f}%',

'Total_Growth%': '{:+.1f}%',

'Deals' : '{:,}',

}))

# Sector diversity over time
```

```
sector_by_year = (df.groupby(['Year', 'Sector'])
.size()
.reset_index(name='Count'))
top_sec5 = df['Sector'].value_counts().head(5).index
plot_df = sector_by_year[sector_by_year['Sector'].isin(top_sec5)]

fig, ax = plt.subplots(figsize=(12, 5))
for sec in top_sec5:
sub = plot_df[plot_df['Sector'] == sec]
ax.plot(sub['Year'], sub['Count'], marker='o',
linewidth=2, label=sec)
ax.set_title('Q2: Top 5 Sectors — Deal Count Growth Over Time',
fontsize=13, fontweight='bold')
ax.set_xlabel('Year')
ax.set_ylabel('Number of Deals')
ax.set_xticks([2018, 2019, 2020, 2021])
ax.legend()
plt.tight_layout()
plt.savefig('Q2_ecosystem_change.png', dpi=150, bbox_inches='tight')
plt.show()
```

Q3 · Do cities play a major role in attracting funding?

```
# CELL 31 — City concentration analysis
city_agg = (df.groupby('City')['Amount_M']
.agg(['sum', 'count'])
.rename(columns={'sum':'Total_M', 'count':'Deals'})
.sort_values('Total_M', ascending=False))

# Top-3 city concentration
top3_share = city_agg.head(3)['Total_M'].sum() / city_agg['Total_M'].sum()
display(HTML(f'Top 3 cities account for {top3_share*100:.1f}% of total funding'))
display(city_agg.head(10).style.format({'Total_M': '${:,.1f}M',
'Deals': '{:,}'}))

# Pie chart of city funding share
city_top10 = city_agg.head(10)
others = city_agg.iloc[10:]['Total_M'].sum()
```

```
pie_data = pd.concat([city_top10['Total_M'],
pd.Series({'Others': others})])

fig, ax = plt.subplots(figsize=(9, 9))
wedges, texts, autotexts = ax.pie(
pie_data.values,
labels = pie_data.index,
autopct = '%1.1f%%',
startangle = 140,
colors = sns.color_palette('tab20', len(pie_data)),
pctdistance= 0.82)
ax.set_title('Q3: City Share of Total Startup Funding ($M)',
fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('Q3_city_share.png', dpi=150, bbox_inches='tight')
plt.show()
```

Q4 · Which industries are most favored by investors?

```
# CELL 32 — Favored sectors: by deals AND by funding amount
sector_summary = (df.groupby('Sector')
.agg(
Deals = ('Company/Brand', 'count'),
Total_M = ('Amount_M', 'sum'),
Median_M = ('Amount_M', 'median'),
)
.sort_values('Total_M', ascending=False)
.head(12))

display(HTML('Q4 — Investor-Favored Sectors'))
display(sector_summary.style
.format({'Total_M':'${:,.1f}M', 'Median_M':'${:,.2f}M',
'Deals':'{:,}'})
.bar(subset=['Total_M'], color='#1B7F79'))

# Bubble chart: Deals (x) vs Total Funding (y), bubble size = Median
fig, ax = plt.subplots(figsize=(12, 7))
scatter = ax.scatter(
```

```
    x = sector_summary['Deals'],
    y = sector_summary['Total_M'],
    s = sector_summary['Median_M'].clip(lower=0.1) * 5,
    c = range(len(sector_summary)),
    cmap = 'tab20',
    alpha = 0.8,
    edgecolors = 'white', linewidths=1.5)

    for _, row in sector_summary.iterrows():
    ax.annotate(row.name,
    (row['Deals'], row['Total_M']),
    fontsize=8.5, ha='center', va='bottom',
    xytext=(0, 5), textcoords='offset points')

    ax.set_xlabel('Number of Deals', fontsize=11)
    ax.set_ylabel('Total Funding ($M)', fontsize=11)
    ax.set_title('Q4: Sectors — Deal Volume vs Total Funding\n'
    '(Bubble size = Median Deal Size)',
    fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.savefig('Q4_sector_bubble.png', dpi=150, bbox_inches='tight')
    plt.show()
```

Q5 · What funding stages dominate the ecosystem?

```
    # CELL 33 — Stage dominance
    stage_sum = (df.groupby('Stage')
    .agg(Deals=('Company/Brand','count'),
    Total_M=('Amount_M','sum'))
    .sort_values('Deals', ascending=False)
    .head(10))

    display(HTML('Q5 — Funding Stage Dominance'))
    display(stage_sum.style.format({'Total_M':'${:,.1f}M', 'Deals':'{:,}'}))

    fig, ax = plt.subplots(figsize=(10, 5))
    ax2 = ax.twinx()

    x = range(len(stage_sum))
```

```
ax.bar(x, stage_sum['Deals'], color='#3B82F6', alpha=0.75, label='Deals')
ax2.plot(x, stage_sum['Total_M'], color='#E8A838', marker='D',
linewidth=2.5, label='Total Funding ($M)')

ax.set_xticks(x)
ax.set_xticklabels(stage_sum.index, rotation=30, ha='right')
ax.set_ylabel('Number of Deals', color='#3B82F6')
ax2.set_ylabel('Total Funding ($M)', color='#E8A838')
ax.set_title('Q5: Funding Stage — Deal Count vs Total Capital',
fontsize=13, fontweight='bold')

lines1, labels1 = ax.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax.legend(lines1 + lines2, labels1 + labels2, loc='upper right')
plt.tight_layout()
plt.savefig('Q5_stage_dominance.png', dpi=150, bbox_inches='tight')
plt.show()
```

Reason: The dual-axis chart (bar + line on two y-axes) elegantly shows both deal count and total capital on one chart. This reveals an important nuance: Seed stage may dominate in deal count but Series C/D may dominate in total capital — two different perspectives on 'dominance'.

PART 8

Preparing Data for Machine Learning

■ EDA is not an end in itself — it informs how you build your ML pipeline. This section translates EDA findings into concrete data preparation steps.

8.1 Feature Engineering

```
# CELL 34 — Create ML-ready features

df_ml = df.copy()

# 1. Company age at time of funding

df_ml['Company_Age'] = df_ml['Year'] - df_ml['Founded']

df_ml['Company_Age'] = df_ml['Company_Age'].clip(lower=0) # no negative ages

# 2. Log-transform the target (Amount_M) to reduce skew

df_ml['log_Amount'] = np.log1p(df_ml['Amount_M'])

# 3. Is the company based in a Tier-1 city?

tier1 = ['Bangalore', 'Mumbai', 'Delhi', 'Gurgaon', 'Hyderabad',

'Chennai', 'Pune', 'Noida']

df_ml['Is_Tier1'] = df_ml['City'].isin(tier1).astype(int)

# 4. Funding round encoded as ordered numeric

stage_order = {'Pre-seed':1, 'Seed':2, 'Pre-series A':3,

'Series A':4, 'Series B':5, 'Series C':6,

'Series D':7, 'Series E':8, 'Series F':9}

df_ml['Stage_Num'] = df_ml['Stage'].map(stage_order)

display(HTML('New ML Features Preview'))

display(df_ml[['Company_Age', 'log_Amount', 'Is_Tier1',

'Stage_Num']].describe())
```

Reason: Raw columns are not ML-ready. The funding stage is ordinal (Seed < Series A < Series B) so mapping it to integers preserves that ordering — better than one-hot encoding which treats all stages as unrelated. log1p on Amount reduces skew so the model doesn't overfit to a handful of mega-deals.

8.2 Encoding Categorical Variables

```
# CELL 35 — One-hot encode low-cardinality categoricals

from sklearn.preprocessing import LabelEncoder
```

```
# One-hot for Sector and City (keep top-N categories, rest = 'Other')

def top_n_encode(series, n=10):

top = series.value_counts().head(n).index

return series.apply(lambda x: x if x in top else 'Other')

df_ml['Sector_clean'] = top_n_encode(df_ml['Sector'], n=10)

df_ml['City_clean'] = top_n_encode(df_ml['City'], n=8)

df_ml = pd.get_dummies(df_ml,

columns=['Sector_clean', 'City_clean'],

drop_first=True,

prefix=['sec', 'city'])

display(HTML(f'Shape after encoding: {df_ml.shape}'))

display(HTML('Encoded Columns'))

display([c for c in df_ml.columns if c.startswith('sec_') or c.startswith('city_')])
```

8.3 Checking Class Imbalance (for Classification Tasks)

```
# CELL 36 — If predicting Stage as target, check class balance

if 'Stage' in df_ml.columns:

stage_dist = df_ml['Stage'].value_counts(normalize=True) * 100

display(HTML('Stage Class Distribution (%)'))

display(stage_dist.round(1).to_frame('% Share'))

# Visualise imbalance

fig, ax = plt.subplots(figsize=(10, 4))

ax.bar(stage_dist.index[:8], stage_dist.values[:8],

color=sns.color_palette('tab10', 8))

ax.set_title('Class Distribution — Funding Stage',

fontsize=12, fontweight='bold')

ax.set_ylabel('% of Records')

plt.xticks(rotation=30, ha='right')

plt.tight_layout()

plt.show()

# If imbalanced: suggest SMOTE or class_weight='balanced'

display(HTML('''

Tip: If Seed class > 40% of records, use
```

```
class_weight="balanced" in sklearn classifiers,
or oversample minority classes with SMOTE.
'''))
```

8.4 Train / Validation / Test Split

```
# CELL 37 — Split data for ML
from sklearn.model_selection import train_test_split

# Define feature matrix X and target y
# Example: predicting log_Amount (regression task)
feature_cols = ['Year', 'Company_Age', 'Is_Tier1', 'Stage_Num'] + \
[c for c in df_ml.columns
if c.startswith('sec_') or c.startswith('city_')]

df_ready = df_ml[feature_cols + ['log_Amount']].dropna()

X = df_ready[feature_cols]
y = df_ready['log_Amount']

# 70% train | 15% validation | 15% test
X_train, X_temp, y_train, y_temp = train_test_split(
X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(
X_temp, y_temp, test_size=0.5, random_state=42)

display(HTML(f'''
**Train:** {len(X_train):,} |
**Validation:** {len(X_val):,} |
**Test:** {len(X_test):,}
'''))
```

Reason: A 70/15/15 split is used instead of the common 80/20 split because our dataset is moderately sized (~2,879 rows). Having a separate validation set allows hyperparameter tuning without leaking test-set information. random_state=42 ensures reproducibility — you get the same split every time you re-run the notebook.

NEXT STEPS — ML Modelling Suggestions

1. **Regression:** Predict Amount_M → try Linear Regression, Random Forest, XGBoost. Evaluate with RMSE and R^2 on log-scale predictions.

2. **Classification:** Predict Stage category → try Logistic Regression, Random Forest, Gradient Boosting. Evaluate with macro F1 (handles imbalance).

3. **Clustering:** Group startups by Sector + City + Stage using K-Means or DBSCAN to find natural investor archetypes.

4. **Feature Importance:** After fitting a tree-based model, plot model.feature_importances_ to validate your EDA findings about which variables matter most.

Quick Reference Cheat Sheet

Task	Key Method	Key Argument / Note
Load CSV	`pd.read_csv()`	dtype=str for mixed-type columns
Quick overview	`df.head(), df.info(), df.describe()`	Use display() not print()
Null map	`msno.matrix(df)`	Visual pattern detection
Rename columns	`df.rename(columns={...})`	Required before pd.concat()
Drop columns	`df.drop(columns=[...], errors='ignore')`	errors='ignore' is safe
Apply function	`df['col'].apply(func)`	For row-by-row transformations
Merge years	`pd.concat([...], ignore_index=True)`	ignore_index resets index
Group & aggregate	`df.groupby('col').agg(...)`	Named aggregations with agg()
Cross-tabulation	`pd.crosstab(row, col)`	Perfect for Stage × Sector
Explode multi-value	`series.str.split(',').explode()`	Use for Investor column
Log transform	`np.log1p(series)`	Handles zeros, reduces skew
Box plot	`sns.boxplot(data, x, y, order)`	Add stripplot for transparency
Heatmap	`sns.heatmap(pivot, annot=True)`	Use fmt='d' for integers
Treemap	`squarify.plot(sizes, label)`	pip install squarify first
Train-test split	`train_test_split(X, y, test_size, random_state)`	Use random_state=42 for reproducibility
Save figure	`plt.savefig('name.png', dpi=150, bbox_inches='tight')`	Always before plt.show()

Indian Startup Funding EDA Guide · Generated for Google Colab · 2018–2021 Dataset · display() approach throughout